AI Hands-on for Developers

From "AI skeptic" to AI-native engineer

Concepts · Q&A · Tips · Hands-on

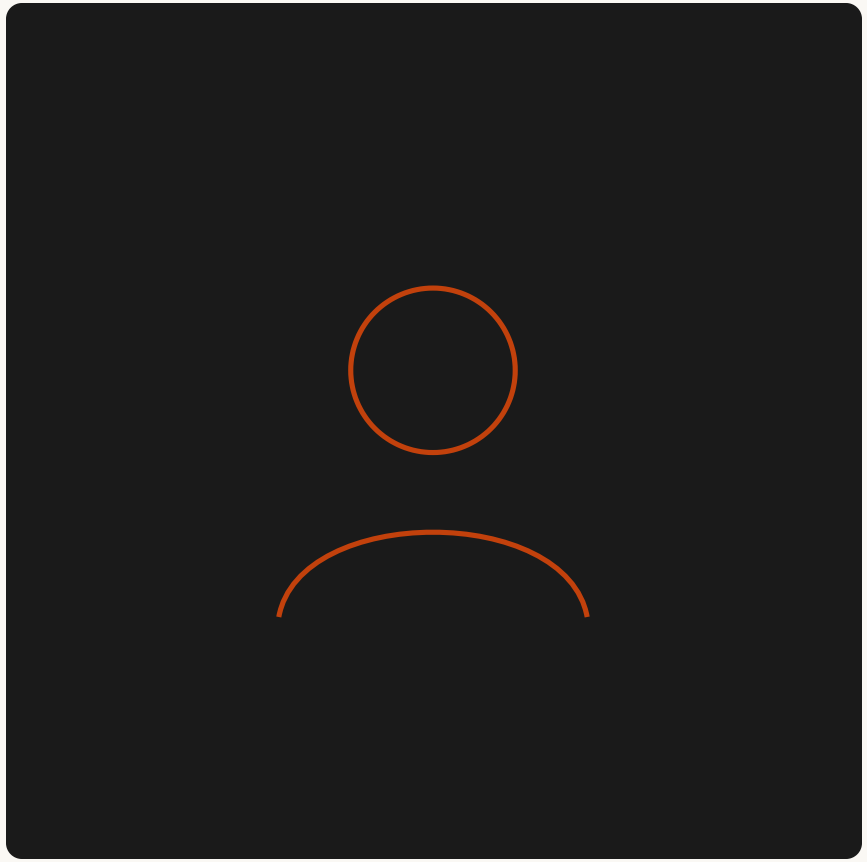
RANDY VROEGOP — RANDY@VROEGOP.ME

Randy Vroegop

Developer · AI-native engineering practice

- Lead Dev modernisation @ Konecranes
- 9 years non-AI programming
- 1 year transition to AI driven development
- Java, Web, AWS, Architecture

randy@vroegop.me · linkedin.com/in/randy-vroegop



What you'll leave with

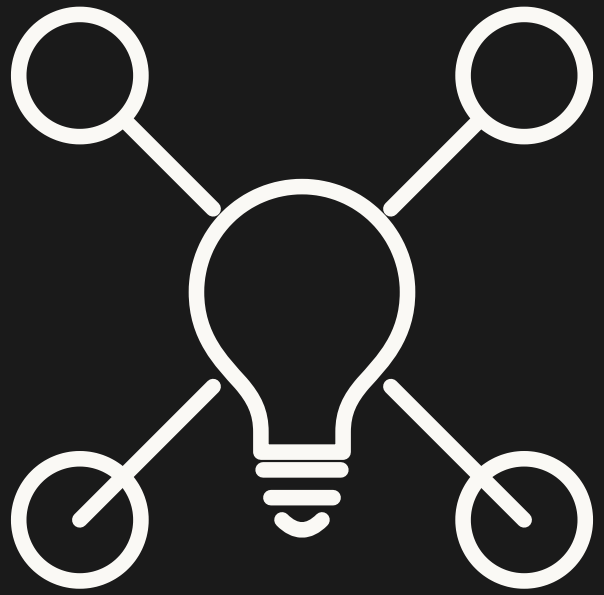
- A mental model for **agents, subagents, MCP, and skills**
- A way to **structure documentation** that agents actually use
- Tools you can use today: **Grill-me-with-docs, GSD, OpenSpec, Superpowers, BTW, Caveman**
- Hands-on exercises you can run on your own laptop
- Honest answers about the **risks** of working this way

Agenda

1. **Concepts** — agents, subagents, MCP, skills, docs
2. **Q&A** — questions that matter, and six honest risks
3. **Tips from the pros** — who to follow, what just shipped
4. **Hands-on** — exercises to run later, on your own laptop

PART 1

Concepts



What is an agent markdown file?

```
---
name: code-reviewer
description: Reviews diffs for correctness bugs.
tools: Read, Grep, Bash
model: sonnet
---

You are a reviewer. Focus on correctness, not style.
When you find a bug, cite file:line. Be terse.
```

Anatomy: frontmatter + body

Frontmatter — config

- `name` — stable identifier
- `description` — *when to use it*
- `tools` — least-privilege list
- `model` — match cost to task

Body — the prompt

- Role and scope
- Hard rules ("never do Z")
- Output format expectations
- Examples of good output

> `description` *isn't human docs — it's how the router decides to delegate.*

Setting up subagent documentation

Two failure modes:

1. **Too vague** — "helps with code". Router can't pick it.
2. **Too greedy** — claims everything. Gets called for tasks it can't do.

A good description answers:

- **Trigger** — what kind of request invokes me?
- **Scope** — what I will and won't do
- **Inputs / Outputs** — what context in, what shape out

When NOT to use a subagent

Subagents aren't free. Each one spawns a fresh context, costs briefing tokens, and hides what really happened.

Skip subagents when:

- The parent already has the context
- You need iterative, conversational work
- You need to *learn* from doing it
- You'd just be passing the same prompt through

> *Delegate **searches and audits**, not **decisions**.*

How agents use MCP servers

MCP = a standard way to expose tools to an agent. Filesystem, GitHub, Jira, Figma, your internal API.

```
// .claude/settings.json
{
  "mcpServers": {
    "github": { "command": "npx", "args": ["-y", "@modelcontextprotocol/server-github"] }
  }
}
```

> *The agent doesn't know GitHub — it knows a tool called* `create_pull_request`

How agents use skills

A **skill** = a reusable, named playbook the agent loads on demand.

Without skills

- One giant system prompt
- All instructions always loaded
- Token bloat
- Conflicting advice

With skills

- Lean default context
- Loaded just in time
- Composable
- Easier to maintain

Teach the agent to build its own skills

When you repeat a workflow → make it a skill.

Prompt pattern:

> *"I just did X three times. Extract it into a skill named `<name>`."*

The agent will:

1. Summarize the repeated steps
2. Write frontmatter with a trigger description
3. Save the playbook — loaded automatically next session

Why documentation matters more, not less

Old world: docs are for humans who forget.
New world: docs are **the runtime context for your agent**.

- Agents can't read your mind, your Slack, or last week's meeting
- Bad docs → confident wrong answers (and you'll trust them)

> *If it's not in the repo, the agent will invent it.*

Structure: router + leaves

Anti-pattern

```
CLAUDE.md  (4,000 lines)
```

- Always loaded
- Burns tokens every turn
- Contradictions pile up
- Nobody updates it

Pattern

```
CLAUDE.md (router, ~100 lines)
docs/
  architecture.md
  testing.md
  domain/
    billing.md
```

What goes in the router

```
# Project router

## Quick facts
- Stack: TypeScript, Postgres, Next.js
- Test: `npm test`   ·   Lint: `npm run lint`

## Where to look
- Architecture decisions → `docs/architecture.md`
- Testing conventions   → `docs/testing.md`
- Domain: billing       → `docs/domain/billing.md`

## House rules
- No `any` in TS
- Prefer composition over inheritance
```

Small, stable, **tells the agent where to read next.**

Why not one big file?

- Every token in always-loaded context is paid **every turn**
- Long context degrades reasoning ("lost in the middle")
- Agents skim
- Humans can review small docs; nobody reviews 4,000-line ones

Tool tour: six to know

Tool	What it does
Grill-me-with-docs	Forces the agent to *interview* you before writing code
Get Shit Done (GSD)	Opinionated workflow for shipping a task end-to-end
OpenSpec	Spec-first development; agent works from a spec doc
Superpowers	A curated bundle of high-quality skills
BTW (`/btw`)	Side-channel session info, no context poisoning
Caveman (`/caveman`)	Ultra-compressed mode — cuts tokens ~75%, accuracy intact

Grill-me-with-docs

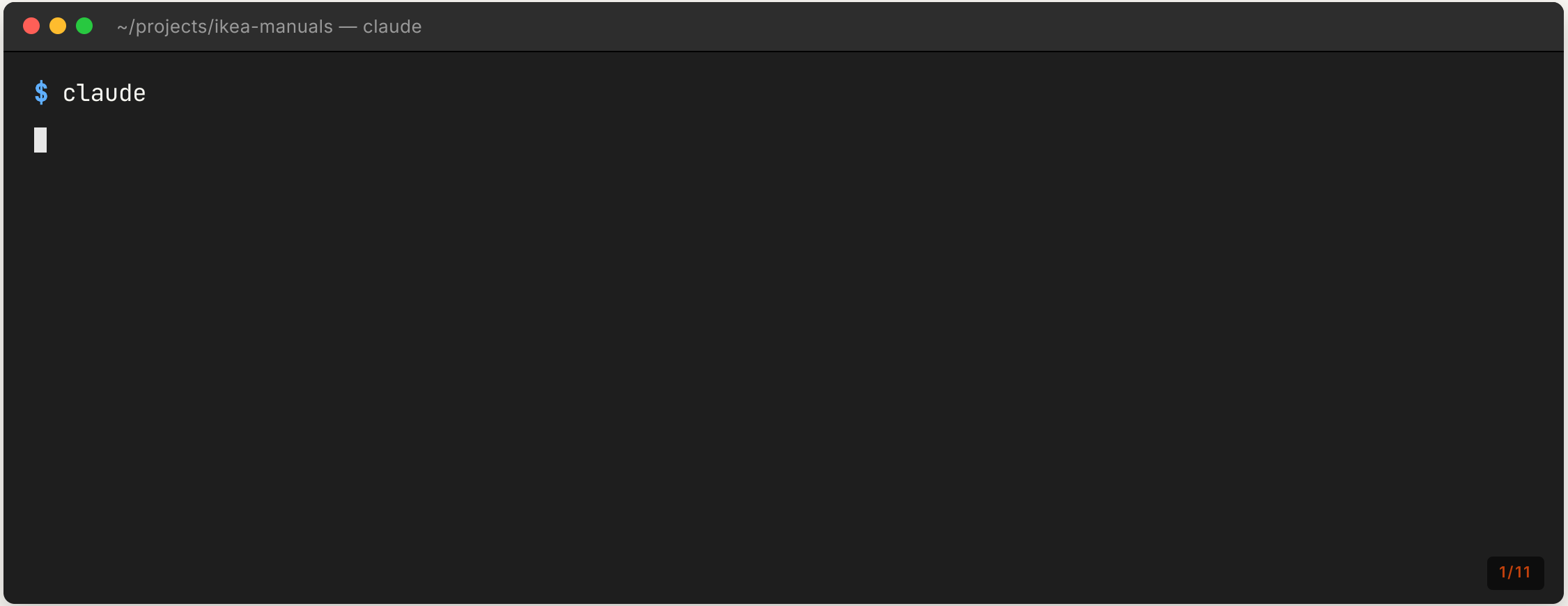
Problem: vague prompt → AI fills the gaps with assumptions → wrong code.

Fix: the skill makes the agent ask the questions a senior engineer would ask, *before* any code:

- "What's the failure mode you're worried about?"
- "Is this a hot path?"
- "Who else writes to this table?"

You write down what you actually want. The code follows from that.

Grill-me-with-docs — what you'd see



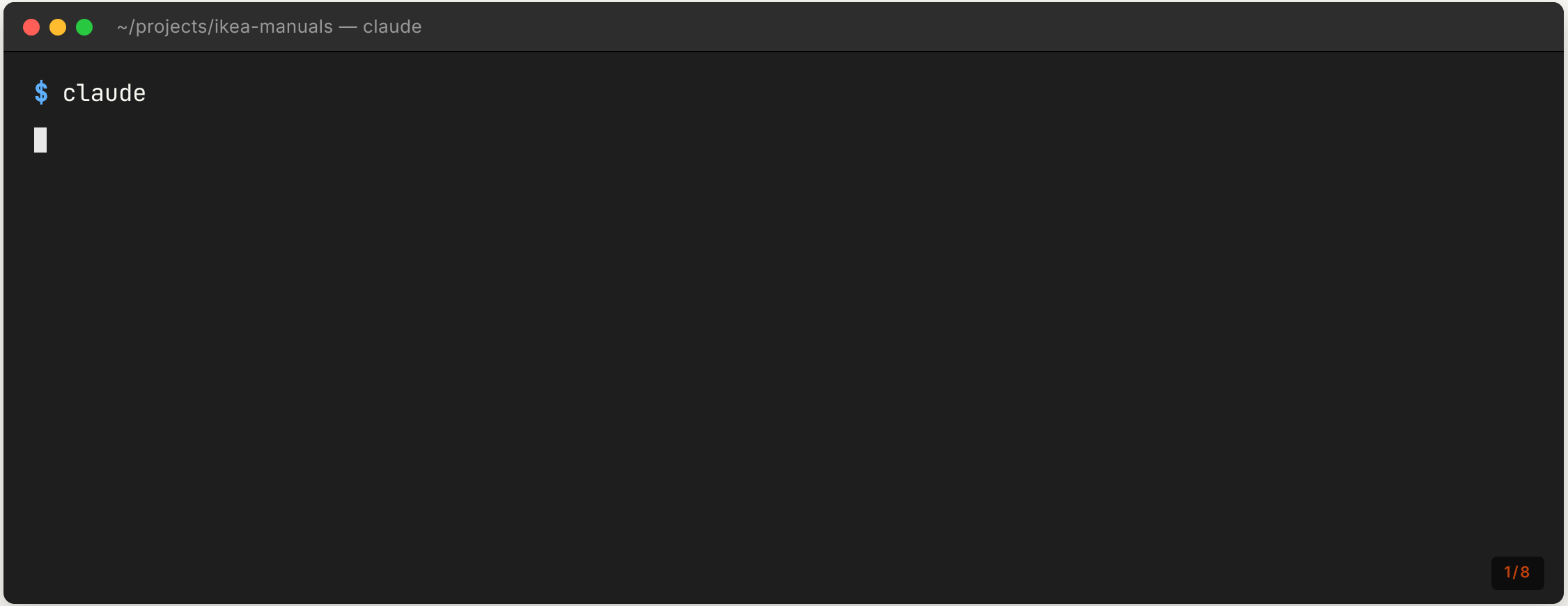
Get Shit Done (GSD)

A workflow skill that drives a task to completion:

1. Restate the task
2. Identify files to touch
3. Make the change
4. Run tests
5. Self-review the diff
6. Commit with a real message

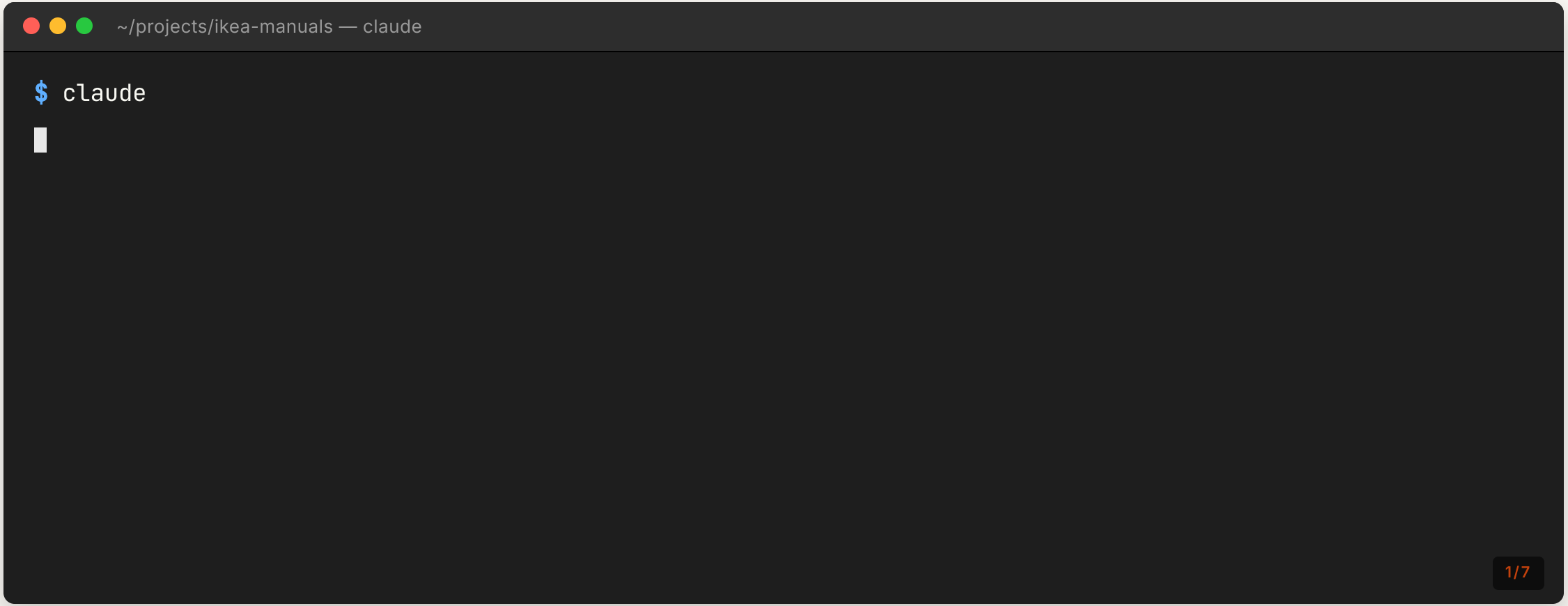
Opinionated. Predictable. Great for chores and small features.

GSD — no spec? write one first



> *The friction is the feature. The spec is reviewable; the chat isn't.*

GSD — implementing from a spec



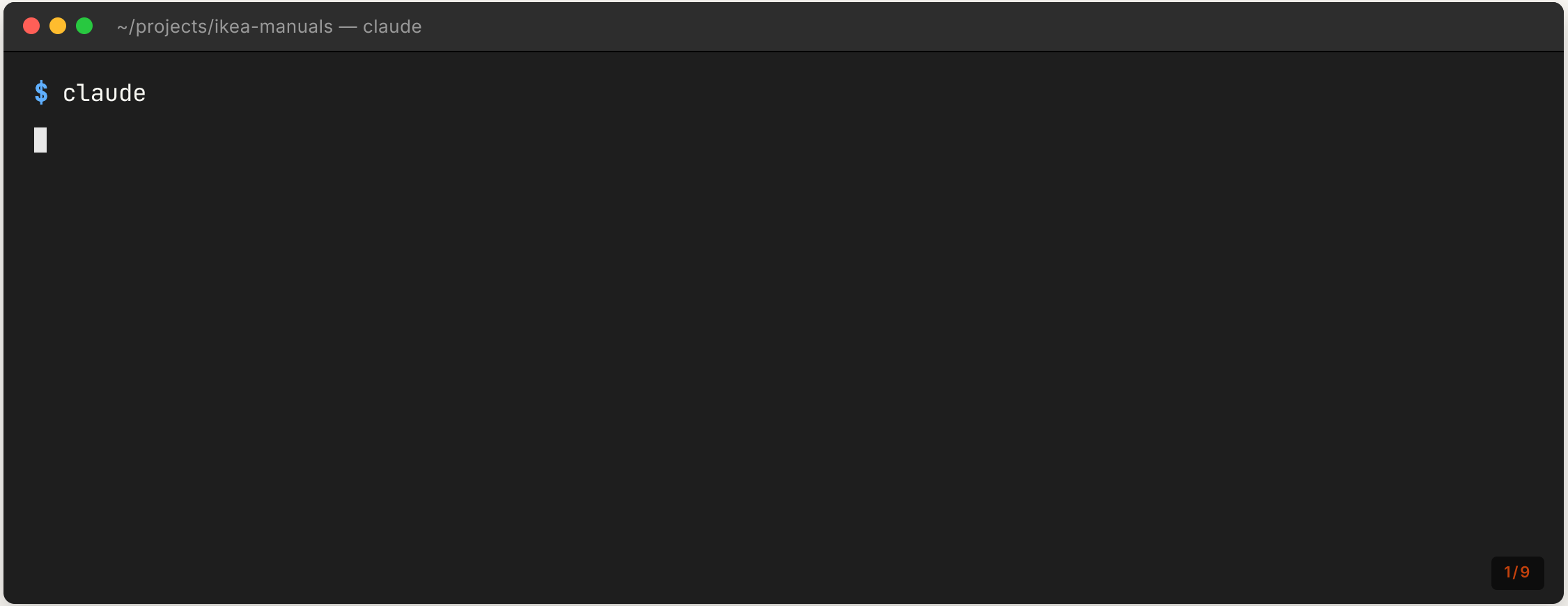
OpenSpec

Spec-first. You write (or generate) a spec markdown, then the agent implements **against the spec, not the chat**.

- Spec is reviewable, diffable, version-controlled
- Stable target — chat drift doesn't change scope
- When the spec is wrong, you fix the spec, not the code

Great for: anything you'd put in a ticket.

OpenSpec — what you'd see



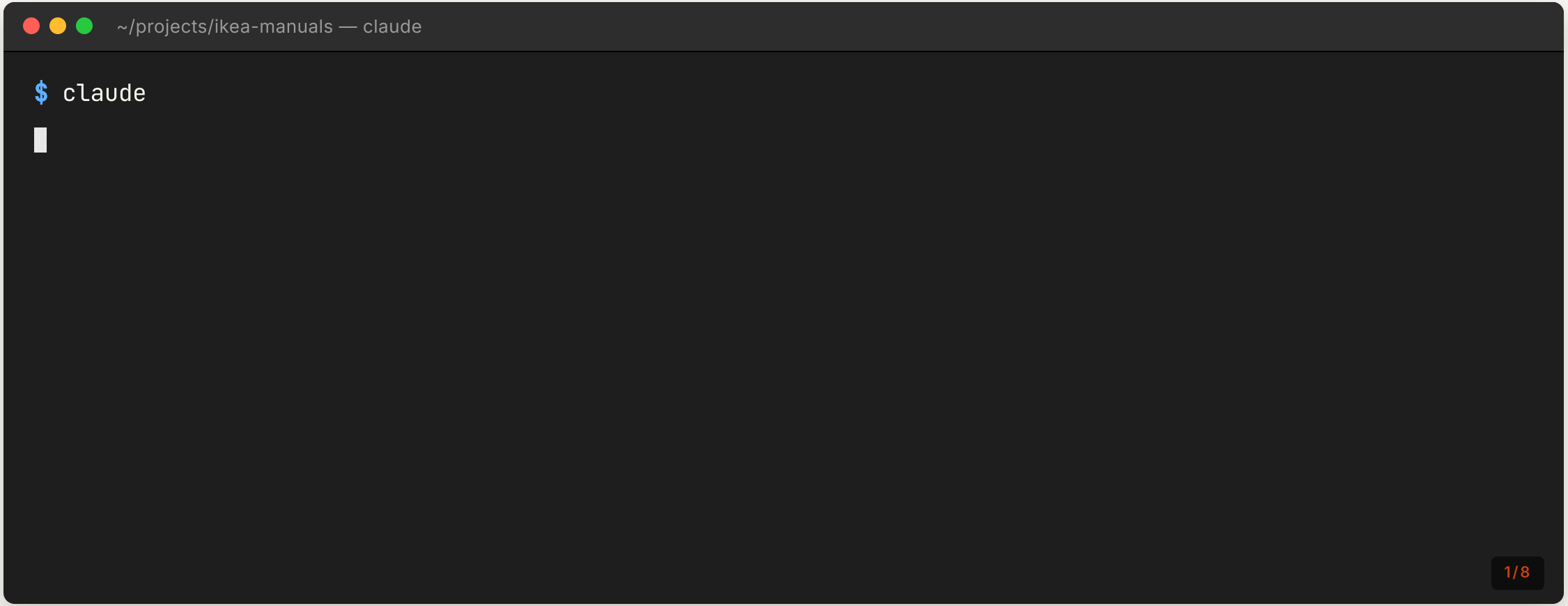
Superpowers

A curated **skill pack**.

- Skills for planning, exploration, verification
- Skills for security review
- Skills for multi-agent fan out
- Skills for refactoring chores

Install once, every project benefits.

Superpowers — what you'd see

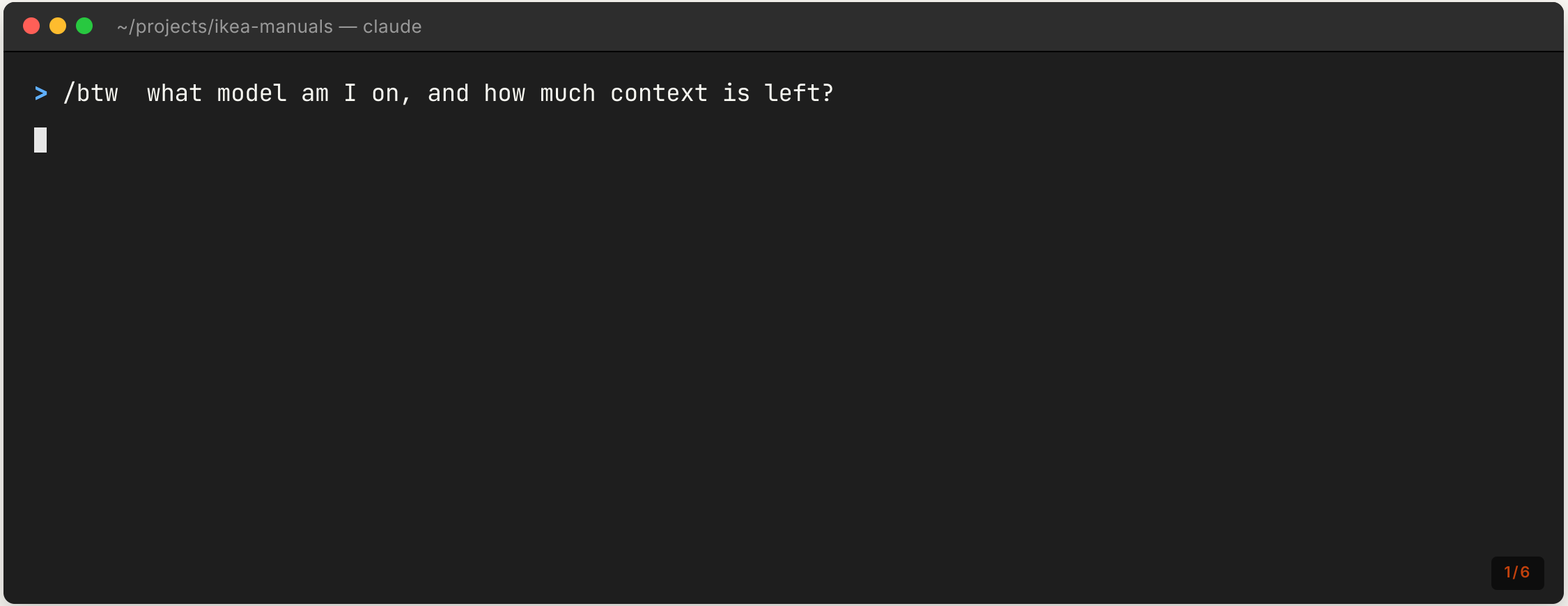


BTW (`/btw`)

"By the way..."

- "Why did you pick that option"
- "Did you already do X"
- "Did I forget to instruct Y"
- "Should we even continue if I want to achieve Z or should I improve the instructions"

BTW — what you'd see



Caveman (`/caveman`)

Ultra-compressed mode. Same technical accuracy, **~75% fewer tokens**.

- Drops articles, hedging, pleasantries
- Keeps file paths, function names, error messages verbatim
- Best for: long sessions, big refactors, anywhere context is the bottleneck

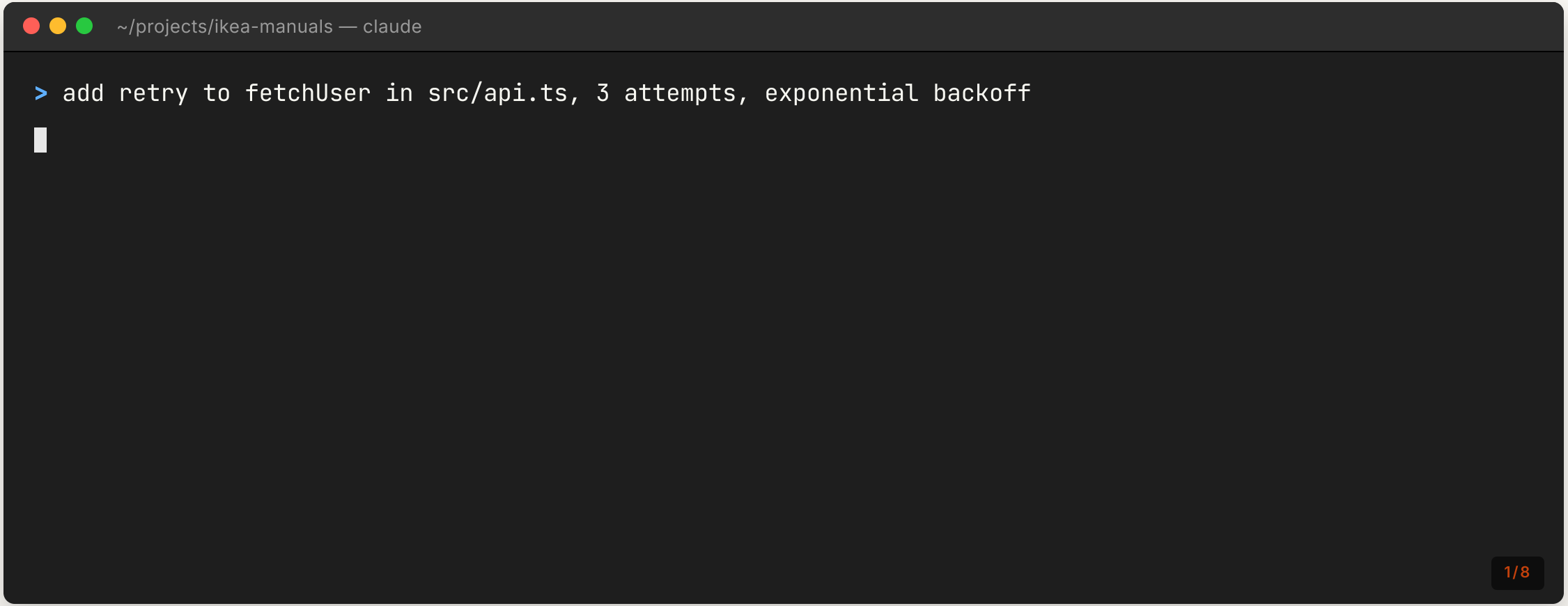
Normal

> *"I'll go ahead and update the `parseRow` function in `src/parser.ts` so it handles empty cells correctly."*

Caveman

> *"Update parseRow `src/parser.ts` — handle empty cells."*

Caveman — what you'd see



Why too much context is bad

- **Cost** — every token paid every turn
- **Latency** — bigger prompt, slower response
- **Quality** — models reason worse on long contexts
- **Confusion** — stale info biases new answers
- **Hidden state** — you lose track of what the model "knows"

> *Context is not a junk drawer. It's working memory.*

Clear the context when...

- You're switching tasks
- The agent is stuck in a wrong assumption
- You're about to do something high-stakes
- The conversation is full of dead ends

> *Re-establishing context is cheap; cleaning up after a confused agent is not.*

Compact the context when...

- You want to keep the *conclusions* but drop the *exploration*
- You're mid-task and approaching limits
- Early turns are no longer relevant but the decisions still are

> *Default: clear more often than you think.*

PART 2

Q&A

The questions that actually matter

Do coding standards still matter?

Short answer: more than ever.

- The agent follows whatever pattern dominates the repo
- No standard → it averages across everything it's ever seen
- Standards are how you steer the agent without prompting every time

> *Let the AI **execute** the standard. Don't let it **invent** it.*

Should we change architecture to suit AI?

A bit, yes — but not by inventing new patterns.

Helps both AI & humans

- Small modules, clear boundaries
- Pure functions where possible
- Explicit dependency injection
- Boring, predictable file layout

Hurts AI

- Magic, reflection, dynamic dispatch
- Implicit conventions
- Mega-files of mixed concerns

Is testing strategy shifting?

Yes. Two real changes:

1. **More tests, lower cost.** Generating tests is cheap; running them is the new bottleneck.
2. **Tests as specs.** Tests become the contract the agent implements against.

> *The agent is great at writing the tests you tell it to write. Mediocre at deciding which tests matter.*

Sharing AI assets — what to commit

Commit to the repo:

- `CLAUDE.md` router
- `docs/*` (leaf docs)
- `.claude/agents/*` (project subagents)
- `.claude/skills/*` (project skills)
- MCP server *configuration* (not secrets)

Do NOT commit: API keys, tokens, personal prefs, local paths, anything in `.claude/settings.local.json`.

How I share knowledge

- Sessions like this
- Hands-on workshops
- Hackathons
- Starter pack repo

> *Teaching teaches me the most.*

How companies usually share (the messy truth)

- Confluence pages nobody reads
- A Slack channel of links nobody bookmarks
- "Ask Bob, he knows"
- Tribal knowledge that walks out with Bob

How companies should now share

- Set up a confluence MCP
- Generate markdown documentation based on Confluence
- Let agents verify information against the codebase - remove what is stale
- "Ask bob" should become "Grill bob" - the AI can fill the gaps
- Verify and read documentation that is going to assist the AI or it will mess up!

Q&A CONTINUED

The risks

Six honest ones — and how to protect yourself

Risk 1 — Feeling fast, shipping less

You feel productive. You ship less.

- Time-to-PR is shorter
- Time-to-merged-and-stable is often **longer**
- Rework, debugging, re-prompting eats the gains

Counter: measure cycle time, track rework rate, treat "looks done" as suspicious until verified.

Risk 2 — Knowing architecture, not code

You stop reading diffs carefully. You drift into being a manager of code.

Fine for some tasks. Dangerous for others.

Counter:

- Read every line for security-, money-, or data-touching code
- Periodically write code from scratch
- Demand the agent explains *why*, not just *what*

Risk 3 — Hallucinations as features

The worst bugs: code that runs, passes tests, and is confidently wrong.

- Made-up API calls that happen to exist
- Plausible-looking math that's subtly off
- "Helpful" extra behavior nobody asked for

Counter: spec-first, tests for risky logic, ask *"what did you change beyond what I asked?"*

Risk 4 — Security gaps

Agents will happily:

- Disable a CSP header to fix a console error
- `chmod 777` to fix permissions
- Skip a hook with `--no-verify`
- Hardcode a "just for testing" secret

Counter: `/security-review` on every diff touching auth/headers/secrets. Pre-commit hooks the agent **cannot** bypass.

Risk 5 — Large blast radius

"Refactor this function" → 47 files changed.

Counter:

- Constrain scope explicitly in the prompt
- `git diff --stat` before accepting
- A skill that rejects diffs touching > N files without confirmation
- Work from clean worktrees, read the diff

Risk 6 — Bad structure costs real money

Every prompt loads context. Bad structure → bigger context → more tokens → more $.

- Monolithic files force loading the whole thing
- Implicit deps force exploratory grep
- Inconsistent naming forces retries

Counter: small files · explicit imports · consistent naming · router docs.

Protecting yourself — the short list

1. **Spec before code** for anything non-trivial
2. **Verify** with the actual app, not just tests
3. **Read diffs** for anything security- or data-critical
4. **Constrain scope** in the prompt
5. **Commit your skills and agents** so the team benefits
6. **Measure cycle time**, not commits

Debugging while pairing with AI

Prompt-only

- Fastest for shallow bugs
- Risky: agent guesses
- Good for: "error X on line Y"

Pairing (REPL)

- You drive hypotheses
- AI runs the experiments
- Good for: unfamiliar code

Test-driven debug — write the failing test first, let AI fix until green. Best for regressions.

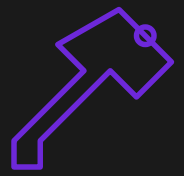
Is your own code easier to debug?

Yes — but the gap is closing.

- **Your code:** you remember the *why*, the trade-offs
- **AI code:** the agent remembers nothing; you read it cold

Mitigations:

- Have the agent leave **decision notes** in the PR description
- Force the agent to write the *failing test first*
- Keep diffs small enough to actually read



PART 3

Hands-on

Do this later — reproduce the demos on your own machine

Setup (5 min)

```
# Clone a sandbox project (any small repo of yours works)
git clone <your-repo>
cd <your-repo>

# Install Claude Code
npm install -g @anthropic-ai/claude-code

# Initialize
claude init
```

You should now have a `.claude/` folder and a `CLAUDE.md`.

Exercise 1 — Grill me with docs

1. Install the `grill-me-with-docs` skill
2. Prompt: *"Add a feature flag system to this project."*
3. Let the agent interview you
4. Save the answers as `docs/specs/feature-flags.md`
5. Then ask the agent to implement against that doc

> *Goal: feel the difference between vibe coding and spec-driven prompts.*

Exercise 2 — GSD vs OpenSpec

Pick a real small task in your repo.

- Do it once with **GSD**
- Reset, do it again with **OpenSpec**

Compare the diffs and the time. Which one would you want for:

- a 30-minute chore?
- a 2-day feature?
- a risky refactor?

Exercise 3 — Install Superpowers

1. Install the Superpowers skill pack
2. Make any small change in your repo
3. Run `/verify` — does it actually launch your app?
4. Run `/code-review` on the diff
5. Run `/security-review`

Notice what each skill is good at — and what it misses.

Exercise 4 — `/btw` discipline

Use `/btw`:

- to check your model
- to check context usage
- to ask "what files have I touched this session?"

> *Train the muscle of asking* ***meta-questions out of band****.*

Exercise 5 — Write your CLAUDE.md router

Replace your `CLAUDE.md` with a router-style file:

- ≤100 lines
- Quick facts (stack, test cmd, lint cmd)
- Routing table to `docs/*`
- House rules

Then move detail into `docs/architecture.md`, `docs/testing.md`, etc.

> *Bonus: ask the agent to do this for you.*

Exercise 6 — Router-style docs

Take your biggest existing doc (or your README) and split it:

- One **router** page (titles + one-line summary + link)
- N **leaf** pages, each focused

Re-run a typical prompt and compare: tokens used, quality of answer.

Exercise 7 — Write a subagent

Create `.claude/agents/<your-agent>.md`:

```
---
name: pr-describer
description: Writes PR descriptions from a diff.
tools: Bash, Read
---

You take the current git diff and produce:
- One-line title (<70 chars)
- Bulleted summary (3 bullets max)
- Test plan checklist
```

Use it on a PR. Iterate until invocation feels reliable.

Thank you

Questions?

randy@vroegop.me · linkedin/randy-vroegop · github/vroegop

Slides, hands-on exercises and skill files live in this repo.